

《操作系统》

实验指导书

指导教师：桑庆兵

实验一、进程调度实验

一、 目的要求

用高级语言编写和调试一个进程调度程序，以加深对进程的概念及进程调度算法的理解。

二、 例题：

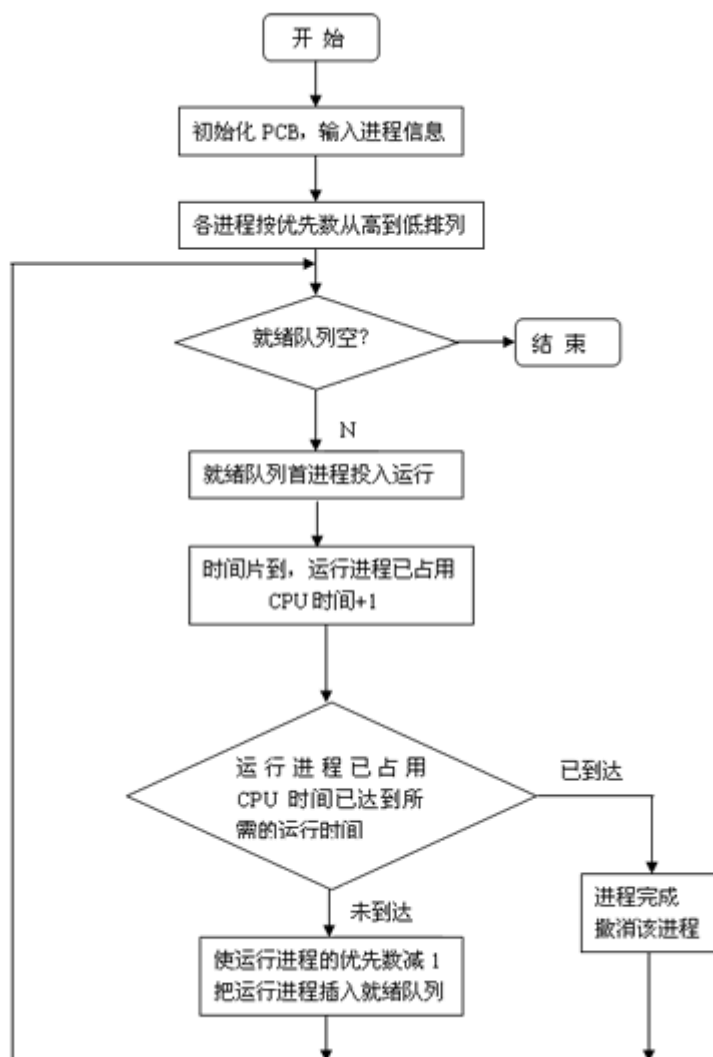
设计一个有 N 个进程共行的进程调度程序。

进程调度算法：采用最高优先数优先的调度算法（即把处理机分配给优先数最高的进程）和先来先服务算法。每个进程有一个进程控制块（PCB）表示。进程控制块可以包含如下信息：进程名、优先数、到达时间、需要运行时间、已用 CPU 时间、进程状态等等。

进程的优先数及需要的运行时间可以事先人为地指定（也可以由随机数产生），进程的到达时间为进程输入的时间，进程的运行时间以时间片为单位进行计算。每个进程的状态可以是就绪 W（Wait）、运行 R（Run）、或完成 F（Finish）三种状态之一（这是编程用到的三个模拟状态，并非进程的三基态）。就绪进程获得 CPU 后都只能运行一个时间片，用已占用 CPU 时间加 1 来表示。如果运行一个时间片后，进程的已占用 CPU 时间已达到所需要的运行时间，则撤消该进程，如果运行一个时间片后进程的已占用 CPU 时间还未达所需要的运行时间，也就是进程还需要继续运行，此时应将进程的优先数减 1（即降低一级），然后把它插入就绪队列等待 CPU。

每进行一次调度程序都打印一次运行进程、就绪队列、以及各个进程的 PCB，以便进行检查。重复以上过程，直到所要进程都完成为止。

调度算法的流程图如下：



进程调度源程序如下：

jingchendiaodu.cpp

```
#include "stdio.h"
```

```
#include <stdlib.h>
```

```
#include <conio.h>
```

```
#define getpch(type) (type*)malloc(sizeof(type))
```

```
#define NULL 0
```

```
struct pcb { /* 定义进程控制块 PCB */
```

```
char name[10];
```

```
char state;

int super;

int ntime;

int rtime;

struct pcb* link;

}*ready=NULL, *p;

typedef struct pcb PCB;

sort() /* 建立对进程进行优先级排列函数*/

{

PCB *first, *second;

int insert=0;

if((ready==NULL) || ((p->super)>(ready->super))) /*优先级最大者, 插入队首*/

{

p->link=ready;

ready=p;

}

else /* 进程比较优先级, 插入适当的位置中*/

{

first=ready;

second=first->link;

while(second!=NULL)

{

if((p->super)>(second->super)) /*若插入进程比当前进程优先数大,*/

{ /*插入到当前进程前面*/
```

```
p->link=second;

first->link=p;

second=NULL;

insert=1;

}

else /* 插入进程优先数最低,则插入到队尾*/

{

first=first->link;

second=second->link;

}

}

if(insert==0) first->link=p;

}

}

input() /* 建立进程控制块函数*/

{

int i,num;

system("cls"); /*清屏*/

printf("\n 请输入进程数?");

scanf("%d",&num);

for(i=0;i<num;i++)

{

printf("\n 进程号 No.%d:\n",i);

p=getpch(PCB);
```

```
printf("\n 输入进程名:");

scanf("%s", p->name);

printf("\n 输入进程优先数:");

scanf("%d", &p->super);

printf("\n 输入进程运行时间:");

scanf("%d", &p->ntime);

printf("\n");

p->rtime=0; p->state='w';

p->link=NULL;

sort(); /* 调用 sort 函数*/

}

}

int space()

{

int l=0; PCB* pr=ready;

while(pr!=NULL)

{

l++;

pr=pr->link;

}

return(l);

}

disp(PCB * pr) /*建立进程显示函数,用于显示当前进程*/

{
```

```
printf("\n qname \t state \t super \t ndtime \t runtime \n");

printf("|%s\t", pr->name);

printf("|%c\t", pr->state);

printf("|%d\t", pr->super);

printf("|%d\t", pr->ntime);

printf("|%d\t", pr->rtime);

printf("\n");

}

check() /* 建立进程查看函数 */

{

PCB* pr;

printf("\n ***** 当前正在运行的进程是:%s", p->name); /*显示当前运行进程*/

disp(p);

pr=ready;

printf("\n *****当前就绪队列状态为:\n"); /*显示就绪队列状态*/

while(pr!=NULL)

{

disp(pr);

pr=pr->link;

}

}

destroy() /*建立进程撤消函数(进程运行结束,撤消进程)*/

{

printf("\n 进程 [%s] 已完成.\n", p->name);
```

```
free(p);

}

running() /* 建立进程就绪函数(进程运行时间到,置就绪状态*/

{

    (p->rtime)++;

    if(p->rtime==p->ntime)

        destroy(); /* 调用 destroy 函数*/

    else

    {

        (p->super)--;

        p->state='w';

        sort(); /*调用 sort 函数*/

    }

}

main() /*主函数*/

{

    int len,h=0;

    char ch;

    input();

    len=space();

    while((len!=0)&&(ready!=NULL))

    {

        ch=getchar();

        h++;

    }

}
```



```
printf("\n The execute number:%d \n",h);

p=ready;

ready=p->link;

p->link=NULL;

p->state='R' ;

check();

running();

printf("\n 按任一键继续.....");

ch=getchar();

}

printf("\n\n 进程已经完成. \n");

ch=getchar();

}
```

三. 实验题:

1 编写并调试一个模拟的进程调度程序，采用“最高优先数优先”调度算法对五个进程进行调度。

“最高优先数优先”调度算法的基本思想是把 CPU 分配给就绪队列中优先数最高的进程。静态优先数是在创建进程时确定的，并在整个进程运行期间不再改变。

动态优先数是指进程的优先数在创建进程时可以给定一个初始值，并且可以按一定原则修改优先数。例如：在进程获得一次 CPU 后就将其优先数减少 1，或者，进程等待的时间超过某一时限时增加其优先数的值，等等。

2 编写并调试一个模拟的进程调度程序，采用“轮转法”调度算法对五个进程进行调度。

轮转法可以是简单轮转法、可变时间片轮转法，或多队列轮转法。

简单轮转法的基本思想是：所有就绪进程按 FCFS 排成一个队列，总是把处理机分配给队首的进程，各进程占用 CPU 的时间片相同。如果运行进程用完它的时间片后还未完成，就把它送回到就绪队列的末尾，把处理机重新分配给队首的进程，直至所有的进程运行完毕。

实验二、作业调度实验

一．目的要求：

用高级语言编写和调试一个或多个作业调度的模拟程序，以加深对作业调度算法的理解。

二．例题：

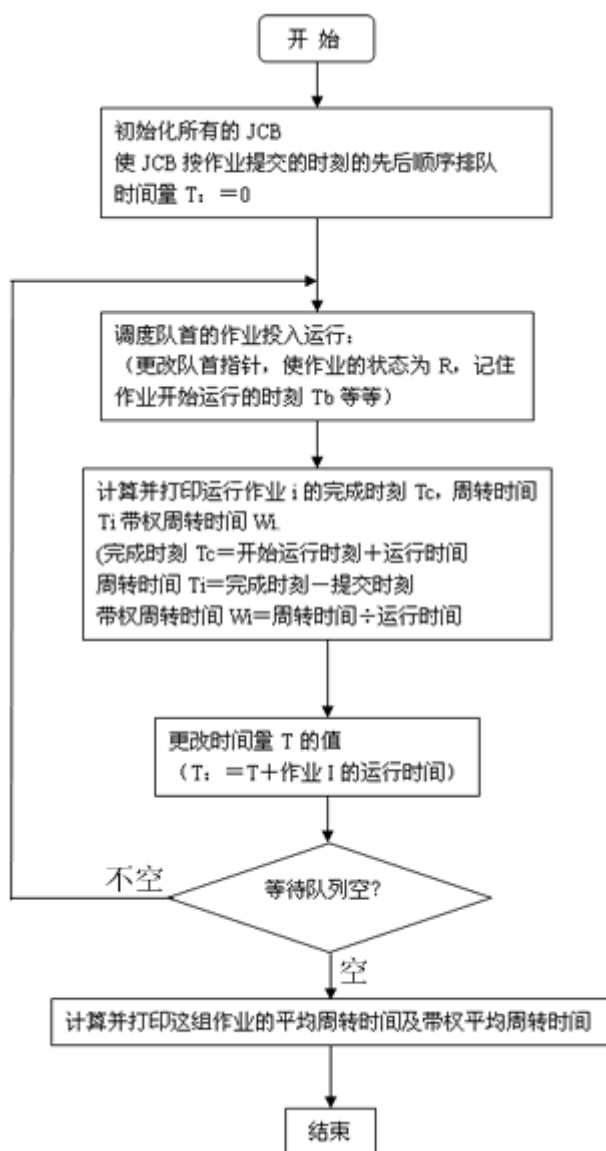
为单道批处理系统设计一个作业调度程序。

由于在单道批处理系统中，作业一投入运行，它就占有计算机的一切资源直到作业完成为止，因此调度作业时不必考虑它所需要的资源是否得到满足，它所占用的 CPU 时限等因素。

作业调度算法：采用先来先服务（FCFS）调度算法，即按作业提交的先后次序进行调度。总是首先调度在系统中等待时间最长的作业。每个作业由一个作业控制块 JCB 表示，JCB 可以包含如下信息：作业名、提交时间、所需的运行时间、所需的资源、作业状态、链指针等等。

作业的状态可以是等待 W(Wait)、运行 R(Run) 和完成 F(Finish) 三种状态之一。每个作业的最初状态总是等待 W。各个等待的作业按照提交时刻的先后次序排队，总是首先调度等待队列中队首的作业。每个作业完成后要打印该作业的开始运行时刻、完成时刻、周转时间和带权周转时间，这一组作业完成后要计算并打印这组作业的平均周转时间、带权平均周转时间。

调度算法的流程图如下：



三．实习题：

- 1 编写并调试一个单道处理系统的作业等待模拟程序。

作业等待算法：分别采用先来先服务（FCFS），最短作业优先（SJF）、响应比高者优先（HRN）的调度算法。

对每种调度算法都要求打印每个作业开始运行时刻、完成时刻、周转时间、带权周转时间，以及这组作业的平均周转时间及带权平均周转时间，以比较各种算法的优缺点。

- 2 编写并调度一个多道程序系统的作业调度模拟程序。

作业调度算法：采用基于先来先服务的调度算法。可以参考课本中的方法进行设计。

对于多道程序系统，要假定系统中具有的各种资源及数量、调度作业时必须考虑到每个作业的资源要求。

3 编写并调试一个多道程序系统的作业调度模拟程序。

作业调度算法：采用基于优先级的作业调度。

可以参考课本中的例子自行设计。

实验三、存储管理实验

一. 目的要求:

- 1 通过编写和调试存储管理的模拟程序以加深对存储管理方案的理解，熟悉虚存管理的各种页面淘汰算法。
- 2 通过编写和调试地址转换过程的模拟程序以加强对地址转换过程的了解。

二. 例题

设计一个请求页式存储管理方案。并编写模拟程序实现之。产生一个需要访问的指令地址流，它是一系列需要访问的指令的地址。为不失一般性，你可以适当地（用人工指定方法或用随机数产生器）生成这个序列，使得 50% 的指令是顺序执行的，25% 的指令均匀地散布在前地址部分，25% 的地址是均匀地散布在后地址部分。

为简单起见，页面淘汰算法采用 FIFO 页面淘汰算法，并且在淘汰一页时，只将该页在页表中抹去，而不再判断它是否被改写过，也不将它写回到辅存。

 具体的做法可以是：

产生一个需要访问的指令地址流；

指令合适的页面尺寸（例如以 1K 或 2K 为 1 页）；

指定内存页表的最大长度，并对页表进行初始化；

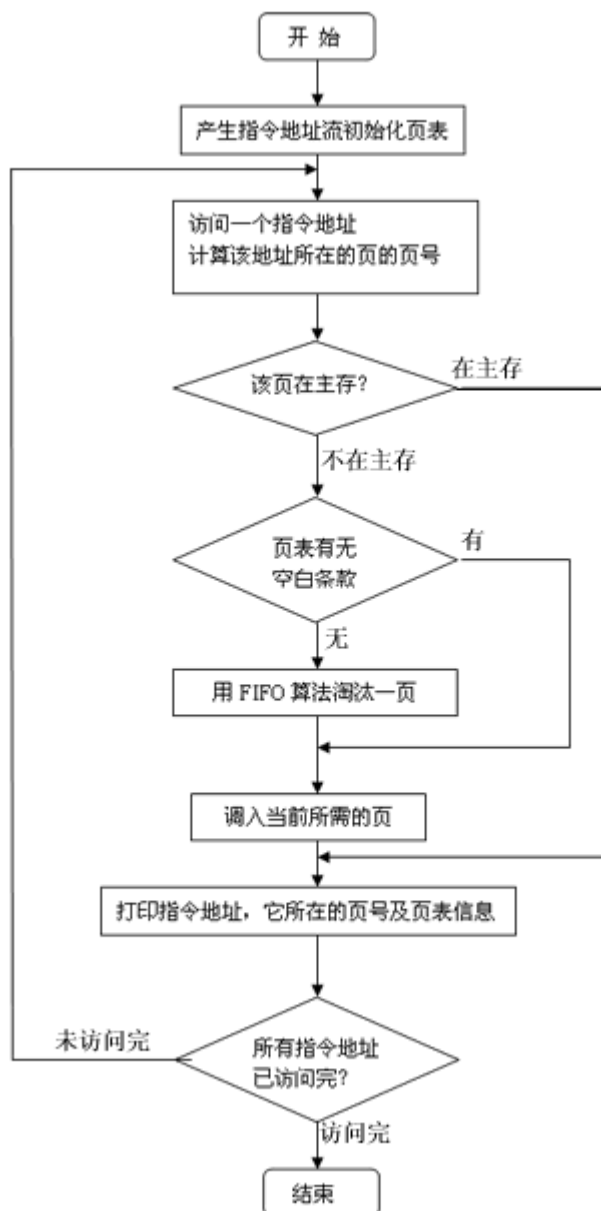
每访问一个地址时，首先要计算该地址所在的页的页号，然后查页表，判断该页是否在主存——如果该页已在主存，则打印页表情况；

如果该页不在主存且页表未满，则调入一页并打印页表情况；

如果该页不在主存且页表已满，则按 FIFO 页面淘汰算法淘汰一页后调入所需的页，打印页表情况；

逐个地址访问，直到所有地址访问完毕。

存储管理算法的流程图如下：



三．实验题：

- 1 设计一个固定式分区分配的存储管理方案，并模拟实现分区的分配和回收过程。

可以假定每个作业都是批处理作业，并且不允许动态申请内存。为实现分区的分配和回收，可以设定一个分区说明表，按照表中的有关信息进行分配，并根据分区的分配和回收情况修改该表。

- 2 设计一个可变式分区分配的存储管理方案，并模拟实现分区的分配和回收过程。

对分区的管理法可以是下面三种算法之一：

- 首次适应算法

● 最坏适应算法

● 最佳适应算法

③ 编写并调试一个段页式存储管理的地址转换的模拟程序。

首先设计好段表、页表，然后给出若干个有一定代表性的地址，通过查找段表页表后得到转换的地址。要求打印转换前的地址，相应的段表，页表条款及转换后的地址，以便检查。

实验四、文件管理实验

一． 目的要求

1 用高级语言编写和调试一个简单的文件系统，模拟文件管理的工作过程，从而对各种文件操作命令的实质内容和执行过程有比较深入的了解。

2 要求设计一个 n 个用户的文件系统，每次用户可保存 m 个文件，用户在一次运行中只能打开一个文件，对文件必须设置保护措施，且至少有 Create、delete、open、close、read、write 等命令。

二． 例题：

设计一个 10 个用户的文件系统，每次用户可保存 10 个文件，一次运行用户可以打开 5 个文件。

程序采用二级文件目录（即设置主目录[MFD]）和用户文件目录（UED）；另外，为打开文件设置了运行文件目录（AFD）。

为了便于实现，对文件的读写作了简化，在执行读写命令时，只需改读写指针，并不进行实际的读写操作

算法与框图：

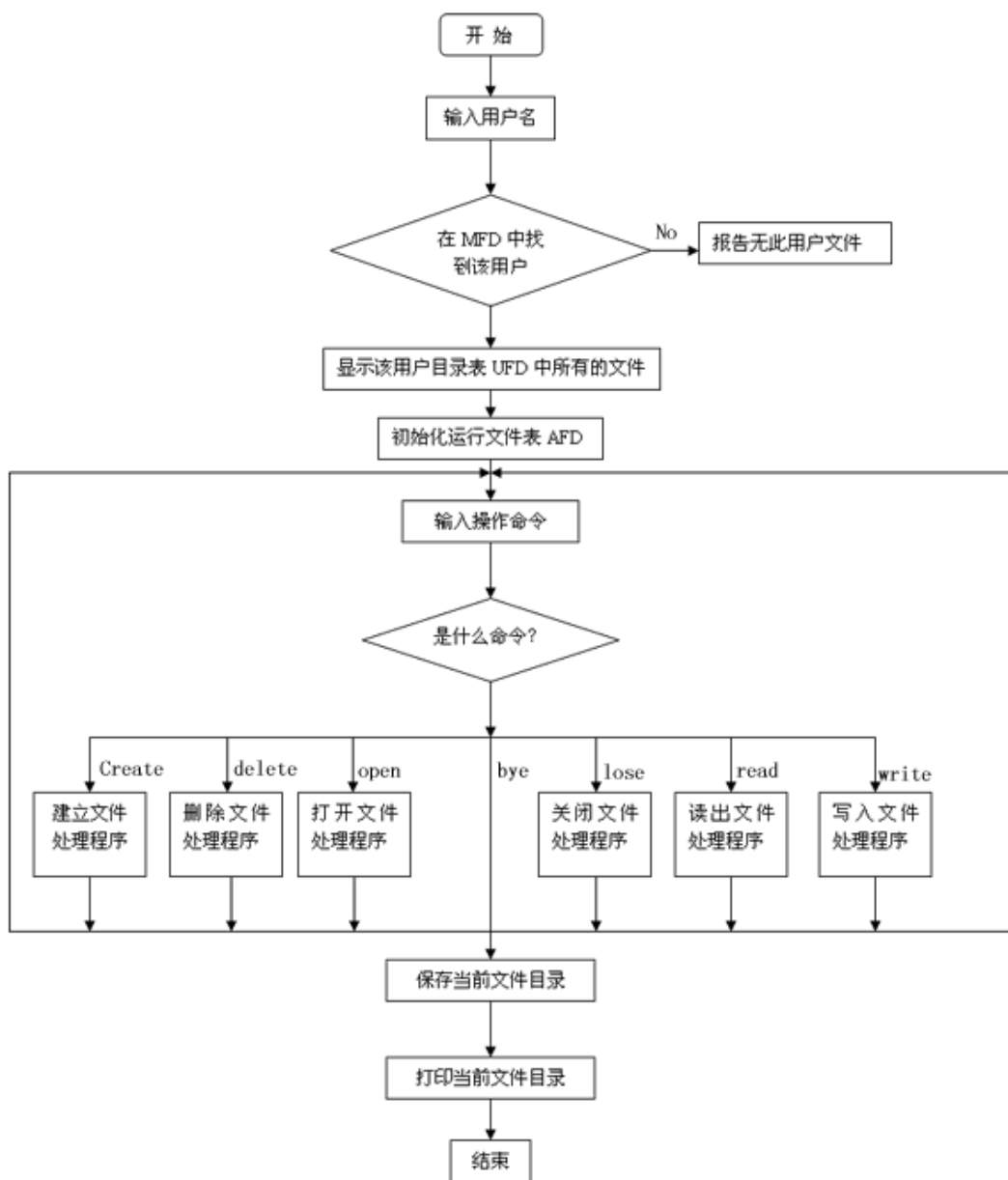
因系统小，文件目录的检索使用了简单的线性搜索。文件保护简单使用了三位保护码：允许读写执行、对应位为 1，对应位为 0，则表示不允许读写、执行。

程序中使用的主要设计结构如下：

主文件目录和用户文件目录（ MFD、UFD）打开文件目录（ AFD）（即运行文件目录）

M D F	U F D	A F D
用户名	文件名	打开文件名
文件目录指针	保护码	打开保护码
用户名	文件长度	读写指针
文件目录指针	文件名	
	•	
	•	
	•	

文件系统算法的流程图如下：



三． 实验题：

- ① 增加 2~3 个文件操作命令，并加以实现（如移动读写指针，改变文件属性，更换文件名，改变文件保护级别）。
- ② 编一个通过屏幕选择命令的文件管理系统，每屏要为用户提供足够的选择信息，不需要打入冗长的命令。

- 3 设计一个树型目录结构的文件系统，其根目录为 root，各分支可以是目录，也可以是文件，最后的叶子都是文件。
- 4 根据学校各级机构，编制一文件系统。

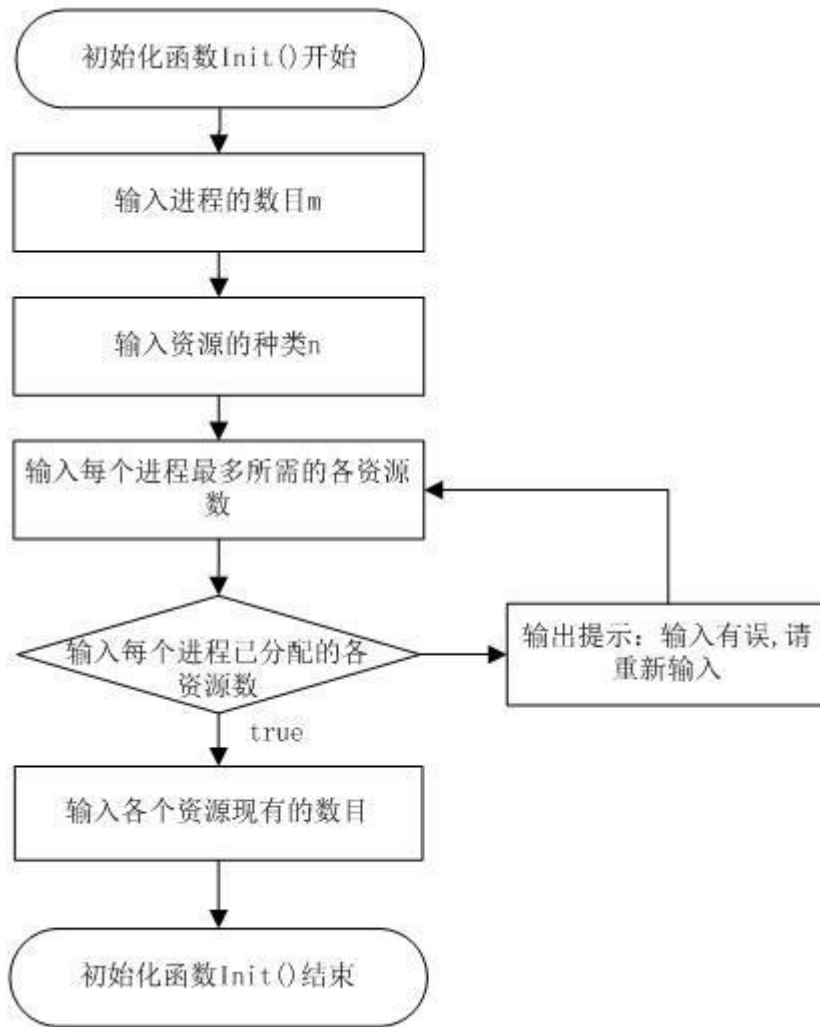
实验五、死锁避免银行家算法实验

程序实现思路银行家算法顾名思义是来源于银行的借贷业务，一定数量的本金要应多个客户的借贷周转，为了防止银行加资金无法周转而倒闭，对每一笔贷款，必须考察其是否能限期归还。在操作系统中研究资源分配策略时也有类似问题，系统中有限的资源要供多个进程使用，必须保证得到的资源的进程能在有限的时间内归还资源，以供其他进程使用资源。如果资源分配不得到就会发生进程循环等待资源，则进程都无法继续执行下去的死锁现象。

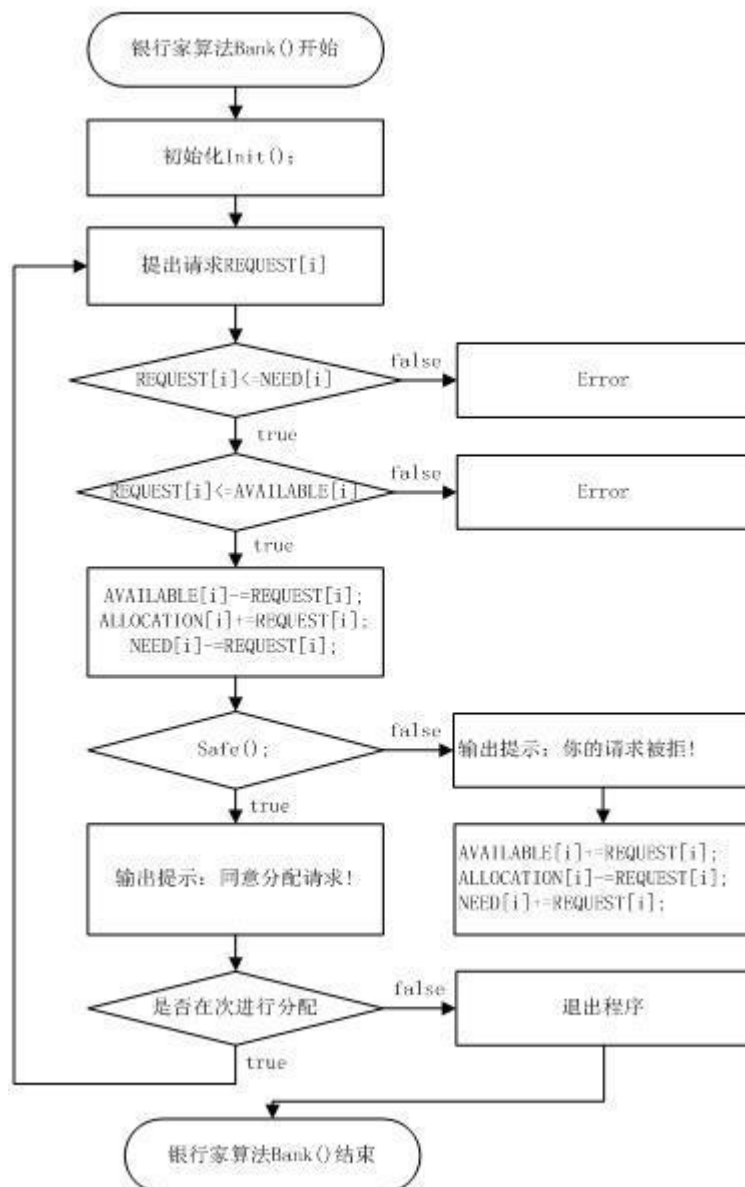
把一个进程需要和已占有资源的情况记录在进程控制中，假定进程控制块 PCB 其中“状态”有就绪态、等待态和完成态。当进程在处于等待态时，表示系统不能满足该进程当前的资源申请。“资源需求总量”表示进程在整个执行过程中总共要申请的资源量。显然，每个进程的资源需求总量不能超过系统拥有的资源总数，银行算法进行资源分配可以避免死锁。

一. 程序流程图:

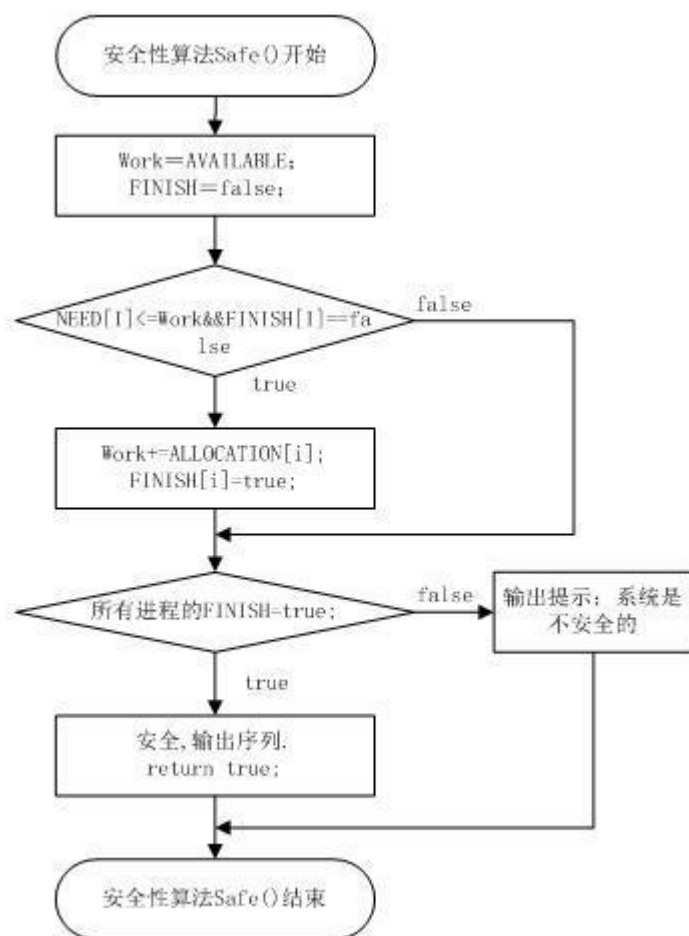
1.初始化算法流程图:



2. 银行家算法流程图:



3.安全性算法流程图:



二. 银行家算法设计:

1. 设进程 i 提出请求 $\text{Request}[n]$, 则银行家算法按如下规则进行判断。

(1)如果 $\text{Request}[n] > \text{Need}[i, n]$, 则报错返回。

(2)如果 $\text{Request}[n] > \text{Available}$, 则进程 i 进入等待资源状态, 返回。

(3)假设进程 i 的申请已获批准, 于是修改系统状态:

$\text{Available} = \text{Available} - \text{Request}$

$\text{Allocation} = \text{Allocation} + \text{Request}$

$\text{Need} = \text{Need} - \text{Request}$

(4)系统执行安全性检查, 如安全, 则分配成立; 否则试探性分配作废, 系统恢复原状, 进程等待。

2. 安全性检查

(1)设置两个工作向量 $\text{Work} = \text{Available}$; $\text{Finish} = \text{False}$

(2)从进程集合中找到一个满足下述条件的进程,

$\text{Finish} = \text{False}$

$\text{Need} \leq \text{Work}$

如找到, 执行(3); 否则, 执行(4)

(3)设进程获得资源, 可顺利执行, 直至完成, 从而释放资源。

$\text{Work} = \text{Work} + \text{Allocation}$

$\text{Finish} = \text{True}$

GO TO 2

(4)如所有的进程 `Finish =true`，则表示安全；否则系统不安全。

3. 数据结构

假设有 M 个进程 N 类资源，则有如下数据结构:

```
#define W 10
#define R 20
int A ;           //总进程数
int B ;           //资源种类
int ALL_RESOURCE[W]; //各种资源的数目总和
int MAX[W] ;      //M 个进程对 N 类资源最大资源需求量
int AVAILABLE ;   //系统可用资源数
int ALLOCATION[W] ; //M 个进程已经得到 N 类资源的资源量
int NEED[W] ;     //M 个进程还需要 N 类资源的资源量
int Request ;     //请求资源个数
```

5.4 主要函数说明

```
void showdata();      //主要用来输出资源分配情况
void changdata(int);  //主要用来输出资源分配后后的情况
void rstordata(int);  //用来恢复资源分配情况,如:银行家算法时,由于分配不安全则要恢复资源分配情况
int chkerr(int);      //银行家分配算法的安全检查
void bank() ;         //银行家算法
```

银行家算法的课程设计 (二)VC++6.02008-01-28 15:29 源程序

数据结构分析:

假设有 M 个进程 N 类资源，则有如下数据结构:

```
#define W 10
#define R 20
int A ;           //总进程数
int B ;           //资源种类
int ALL_RESOURCE[W]; //各种资源的数目总和
int MAX[W] ;      //M 个进程对 N 类资源最大资源需求量
int AVAILABLE ;   //系统可用资源数
int ALLOCATION[W] ; //M 个进程已经得到 N 类资源的资源量
int NEED[W] ;     //M 个进程还需要 N 类资源的资源量
int Request ;     //请求资源个数
```

第 3 章 程序清单#include <iostream>

```
using namespace std;
#define MAXPROCESS 50          /*最大进程数*/
#define MAXRESOURCE 100       /*最大资源数*/
int AVAILABLE[MAXRESOURCE];    /*可用资源数组*/
int MAX[MAXPROCESS][MAXRESOURCE]; /*最大需求矩阵*/
int ALLOCATION[MAXPROCESS][MAXRESOURCE]; /*分配矩阵*/
int NEED[MAXPROCESS][MAXRESOURCE]; /*需求矩阵*/
int REQUEST[MAXPROCESS][MAXRESOURCE]; /*进程需要资源数*/
bool FINISH[MAXPROCESS];       /*系统是否有足够的资源分配*/
int p[MAXPROCESS];             /*记录序列*/
```

```

int m,n;                /*m 个进程,n 个资源*/
void Init();
bool Safe();
void Bank();
int main()
{
    Init();
    Safe();
    Bank();
}
void Init()              /*初始化算法*/
{
    int i,j;
    cout<<"t-----"<<endl;
    cout<<"t|                ||"<<endl;
    cout<<"t|            银行家算法        ||"<<endl;
    cout<<"t|                ||"<<endl;
    cout<<"t|            计科 04151 李宏    ||"<<endl;
    cout<<"t|                ||"<<endl;
    cout<<"t|            0415084211    ||"<<endl;
    cout<<"t-----"<<endl;
    cout<<"请输入进程的数目:";
    cin>>m;
    cout<<"请输入资源的种类:";
    cin>>n;
    cout<<"请输入每个进程最多所需的各资源数,按照"<<m<<"x"<<n<<"矩阵输入"<<endl;
    for(i=0;i<m;i++)
    for(j=0;j<n;j++)
    cin>>MAX[j];
    cout<<"请输入每个进程已分配的各资源数,也按照"<<m<<"x"<<n<<"矩阵输入"<<endl;
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            cin>>ALLOCATION[j];
            NEED[j]=MAX[j]-ALLOCATION[j];
            if(NEED[j]<0)
            {
                cout<<"您输入的第"<<i+1<<"个进程所拥有的第"<<j+1<<"个资源数错误,请重新
输入:"<<endl;
                j--;
                continue;
            }
        }
    }
}

```



```
}
cout<<"请输入各个资源现有的数目:"<<endl;
for(i=0;i<n;i++)
{
    cin>>AVAILABLE;
}
}
void Bank()          /*银行家算法*/
{
    int i,cusneed;
    char again;
    while(1)
    {
        cout<<"请输入要申请资源的进程号(注:第 1 个进程号为 0,依次类推)"<<endl;
        cin>>cusneed;
        cout<<"请输入进程所请求的各资源的数量"<<endl;
        for(i=0;i<n;i++)
        {
            cin>>REQUEST[cusneed];
        }
        for(i=0;i<n;i++)
        {
            if(REQUEST[cusneed]>NEED[cusneed])
            {
                cout<<"您输入的请求数超过进程的需求量!请重新输入!"<<endl;
                continue;
            }
            if(REQUEST[cusneed]>AVAILABLE)
            {
                cout<<"您输入的请求数超过系统有的资源数!请重新输入!"<<endl;
                continue;
            }
        }
        for(i=0;i<n;i++)
        {
            AVAILABLE-=REQUEST[cusneed];
            ALLOCATION[cusneed]+=REQUEST[cusneed];
            NEED[cusneed]-=REQUEST[cusneed];
        }
        if(Safe())
        {
            cout<<"同意分配请求!"<<endl;
        }
        else
```

```
{
    cout<<"您的请求被拒绝!"<<endl;
    for(i=0;i<n;i++)
    {
        AVAILABLE+=REQUEST[cusneed];
        ALLOCATION[cusneed]-=REQUEST[cusneed];
        NEED[cusneed]+=REQUEST[cusneed];
    }
}
for(i=0;i<m;i++)
{
    FINISH=false;
}
cout<<"您还想再次请求分配吗?是请按 y/Y,否请按其它键"<<endl;
cin>>again;
if(again=='y' || again=='Y')
{
    continue;
}
break;
}
}

bool Safe()                /*安全性算法*/
{
    int i,j,k,l=0;
    int Work[MAXRESOURCE];    /*工作数组*/
    for(i=0;i<n;i++)
        Work=AVAILABLE;
    for(i=0;i<m;i++)
    {
        FINISH=false;
    }
    for(i=0;i<m;i++)
    {
        if(FINISH==true)
        {
            continue;
        }
        else
        {
            for(j=0;j<n;j++)
            {
                if(NEED[j]>Work[j])
                {
```

```
        break;
    }
}
if(j==n)
{
    FINISH=true;
    for(k=0;k<n;k++)
    {
        Work[k]+=ALLOCATION[k];
    }
    p[l++]=i;
    i=-1;
}
else
{
    continue;
}
}
if(l==m)
{
    cout<<"系统是安全的"<<endl;
    cout<<"安全序列:"<<endl;
    for(i=0;i<l;i++)
    {
        cout<<p;
        if(i!=l-1)
        {
            cout<<"-->";
        }
    }
    cout<<" "<<endl;
    return true;
}
}
cout<<"系统是不安全的"<<endl;
return false;
}
```

输出数据

```
"E:\银行家算法\Debug\yhj.exe"
开始读入初始化文档
资源个数: 3
可利用资源
资源类型 1 2 3
          4 4 3
进程个数: 5
资源类型 1 2 3
P1:       7 5 3
P2:       3 2 2
P3:       9 0 2
P4:       2 2 2
P5:       4 3 3
读入成功

开始读入已分配资源列表
资源类型 1 2 3
P1:       0 1 0
P2:       2 0 0
P3:       3 0 2
P4:       2 2 1
P5:       0 0 2
读入成功

设置需求矩阵
资源类型 1 2 3
P1:       7 4 3
P2:       1 2 2
P3:       6 0 0
P4:       0 0 1
P5:       4 3 1
设置成功
```